

# Forge Beyond Finite Horizons

Unbounded-stack summaries, recurrent strategies,  
and exact probabilistic termination

Concrete algorithms, dual certificates, source bridges,  
and a reproducible Python implementation

Prepared for Vladimir Reshetnikov

15 September 2026

---

**Scope.** An additive proposal against Forge commit `c98e47c5f804`. It does not repeat the existing proof planner, invariant-closure portfolio, nonlinear workers, or verification architecture.

**Executed.** Three search engines; separate replay code; 36 unit tests; 22,100 exhaustive small game arenas; 28,566 stored certificate records.

**Not executed.** Lean elaboration, Lean checker soundness, source-to-model reification, and comparisons with existing tactics. This package does not install an implemented Lean tactic.

---

## Abstract

Forge already proposes a substantial collection of structural, algebraic, and finite-closure workers. The next useful contribution should therefore be a new class of obligations, rather than another description of its controller. This article develops three such classes: reachability and safety of systems with an unbounded stack; synthesis of a strategy that meets an obligation infinitely often against an adversary; and exact absorption probabilities and expected costs of rational finite-state probabilistic programs.

Each class receives an explicit finite certificate language, a positive and a negative outcome, a terminating search algorithm on a stated fragment, and a separate replay computation. Pushdown reachability uses productive pop-summary DAGs, while nonreachability uses closed exclusion relations. Büchi games use positional policies with either a rank that resets at accepting vertices or a co-Büchi rank that bounds the total number of accepting visits. Markov chains use exact Bellman residuals together with a properness potential; a positive path into a closed nonterminal class certifies failure of almost-sure termination. The mathematical kernels are classical. The contribution is their precise packaging as new Forge capabilities, including source bridges that preserve stack suffixes, ownership and temporal progress, or probability mass.

A dependency-free Python package implements all three workers. Its experiments include 881 pushdown cases, 22,300 game arenas, and 770 Markov chains. All 28,566 stored certificate records replay without importing a search module. The game suite exhausts every total arena with at most three vertices, every ownership map, and every accepting set. A 31-node pushdown certificate describes a run of 2,147,483,647 steps without expanding it. These are prototype results, not Lean proofs or evidence of an advantage over another tactic. The article gives soundness arguments, concrete Lean module contracts, implementation priorities, and explicit acceptance gates for closing that remaining gap.

## How to read this package

The mathematical proposal is in Sections 1–5. Implementation contracts and libraries follow in Section 6. The measurements and their units are in Section 7. Appendices describe the wire format and reproduction commands. The accompanying Python files, rather than pseudocode in this article, are the executable reference. Read `results/summary.json` for the machine-readable evidence and `docs/STATUS.md` before interpreting any result as a formal proof.

# Contents

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                                 | <b>i</b>  |
| <b>1 The actual delta to Forge</b>                                              | <b>1</b>  |
| 1.1 A pinned, deliberately narrow novelty claim . . . . .                       | 1         |
| 1.2 Three semantic quantifiers rather than three solver names . . . . .         | 2         |
| 1.3 What is already executable . . . . .                                        | 2         |
| <b>2 Unbounded stacks: productive summaries and closed exclusions</b>           | <b>2</b>  |
| 2.1 Input language and the question being decided . . . . .                     | 2         |
| 2.2 The finite object that summarizes an infinite configuration space . . . . . | 3         |
| 2.3 Search: indexed semi-naïve saturation . . . . .                             | 4         |
| 2.4 Positive certificate: productivity, not just closure . . . . .              | 5         |
| 2.5 Negative certificate: a closed over-approximation . . . . .                 | 6         |
| 2.6 Regular target languages, with an explicit bottom marker . . . . .          | 6         |
| 2.7 How Forge would use the worker . . . . .                                    | 7         |
| <b>3 Persistent witnesses: Büchi strategies and their dual certificates</b>     | <b>8</b>  |
| 3.1 A strategy is a witness with obligations over its future . . . . .          | 8         |
| 3.2 Positive certificate: decrease until acceptance, then reset . . . . .       | 8         |
| 3.3 Negative certificate: bound all future accepting visits . . . . .           | 9         |
| 3.4 Search by repeated attractors . . . . .                                     | 10        |
| 3.5 Completeness and computational costs . . . . .                              | 11        |
| 3.6 Examples that separate the semantics . . . . .                              | 11        |
| 3.7 A concrete next extension: several recurring obligations . . . . .          | 12        |
| <b>4 Exact probabilistic termination and values</b>                             | <b>12</b> |
| 4.1 The input semantics must include probability mass . . . . .                 | 12        |
| 4.2 Why a harmonic equation is insufficient . . . . .                           | 13        |
| 4.3 The certificate and its local checker . . . . .                             | 13        |
| 4.4 A finite-horizon proof interface with explicit error bounds . . . . .       | 14        |
| 4.5 Search: graph structure first, rational solving second . . . . .            | 15        |
| 4.6 Negative certificate: a reachable closed nonterminal class . . . . .        | 16        |
| 4.7 Worked examples and adversarial distinctions . . . . .                      | 16        |
| 4.8 Extensions that require additional certificates . . . . .                   | 17        |
| <b>5 The source bridges are the integration, not a footnote</b>                 | <b>17</b> |
| 5.1 A checked finite model is not yet a proof of the original goal . . . . .    | 17        |
| 5.2 Reachability: forward simulation and witness lifting differ . . . . .       | 17        |

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 5.3      | Games need alternating, ownership-aware refinement . . . . .               | 18        |
| 5.4      | Stuttering can destroy liveness even when it preserves safety . . . . .    | 19        |
| 5.5      | Probability requires exact pushforward, not support simulation . . . . .   | 19        |
| 5.6      | What to borrow from Leant and Djex, without repeating their work . . . . . | 20        |
| <b>6</b> | <b>A concrete Lean implementation plan</b>                                 | <b>20</b> |
| 6.1      | New modules, not a replacement for the existing controller . . . . .       | 20        |
| 6.2      | The exact theorems to implement first . . . . .                            | 21        |
| 6.3      | Quoting a certificate without expanding its witness . . . . .              | 22        |
| 6.4      | Libraries and what they would actually do . . . . .                        | 22        |
| 6.5      | A router driven by proof obligations . . . . .                             | 23        |
| 6.6      | Interface examples, with unsupported features made visible . . . . .       | 23        |
| <b>7</b> | <b>Experiments, replay, and what the evidence does not say</b>             | <b>24</b> |
| 7.1      | Execution protocol . . . . .                                               | 24        |
| 7.2      | Pushdown reference computations and their limits . . . . .                 | 25        |
| 7.3      | An exhaustive finite-game corpus . . . . .                                 | 25        |
| 7.4      | Exact-chain oracles . . . . .                                              | 26        |
| 7.5      | Negative controls and transport tests . . . . .                            | 26        |
| 7.6      | Search-free replay and performance interpretation . . . . .                | 26        |
| <b>8</b> | <b>Implementation order and falsifiable acceptance gates</b>               | <b>27</b> |
| 8.1      | What should remain outside the first release . . . . .                     | 28        |
| 8.2      | Assessment . . . . .                                                       | 28        |
| <b>A</b> | <b>Wire format and executable entry points</b>                             | <b>29</b> |
| <b>B</b> | <b>Reproduction and artifact map</b>                                       | <b>29</b> |
| <b>C</b> | <b>A compact theorem-obligation ledger</b>                                 | <b>30</b> |
|          | <b>References</b>                                                          | <b>31</b> |

# 1 The actual delta to Forge

## 1.1 A pinned, deliberately narrow novelty claim

The reference point is Forge commit `c98e47c5f804e92880fc1d0e378c1b95832b685c`. Its README and merged provenance ledger describe eighteen proposals and an extensive existing capability set [1, 2]. This article was prepared after reading that merged ledger, its conclusions, the Leant/Djex integration notes, and the neighboring projects’ published interfaces. Targeted repository searches supplemented that inspection. It is not a byte-by-byte audit of every file in every frozen submission, and the claim of nonduplication is relative to the published merged design and capability ledger.

In particular, induction with generalization, finite-algebra covers, polynomial and observable-space closure, exact arithmetic witnesses, Kripke countermodels, telescoping, Ore transport, cyclic proof compilation through well-founded measures, and the certificate acceptance boundary are already baseline material. None is presented here as a new invention. The baseline’s own conclusion correctly asks for narrow implementations instead of further vague architecture [3]. This proposal follows that direction by shipping three complete Python search/replay pairs and by stating the precise Lean theorems still required.

The new mathematical ingredients are also not original discoveries in the literature: pushdown saturation, finite Büchi games, and finite-chain absorption are established subjects. What is new to the inspected Forge design is a set of *source-level obligations*, *specialized certificate formats*, and *implemented adapters and tests* for these subjects. The article makes no claim of worldwide priority for the algorithms or rank arguments.

| Existing prerequisite                 | Added capability                                | Distinction that must survive translation                                                                          |
|---------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Finite-algebra closure                | Pushdown reachability and regular stack targets | A finite summary describes infinitely many configurations; it is not a finite enumeration of those configurations. |
| Well-founded cyclic proof compilation | Recurrent adversarial strategies                | Execution may continue forever. A rank resets after acceptance rather than decreasing globally.                    |
| Weighted finite-word observations     | Infinite-horizon probabilistic values           | A harmonic equation alone does not prove absorption, uniqueness, or preservation of probability mass.              |
| Typed witness synthesis               | Strategies with persistent behavioral laws      | One policy must work against every future opponent choice, not one unrelated witness per finite sample.            |

Table 1: The extension boundary. Baseline mechanisms can be reused as infrastructure; they are not claimed to be unable to encode these problems in principle.

There is an important qualification to the first row. Pop summaries can be viewed as a finite relational closure, and derivations can be encoded by trees. Consequently Forge’s generic cover or closure principles may be useful in their formalization. The addition is not “finite closure, again.” It is the suffix-parametric pushdown semantics, its exact translation, its productive proof representation, its regular-target compiler, and the resulting complete worker for a named infinite-state fragment. The same distinction applies to reusing ordinary arithmetic to check ranks and rational residuals.

## 1.2 Three semantic quantifiers rather than three solver names

Consider a program that can remain in a loop or leave it. Depending on the meaning of a transition, three entirely different questions arise:

$$\begin{aligned} &\exists \text{ a finite execution reaching a target;} \\ &\exists \sigma \forall \text{ opponent choices } \mathbf{GF} B; \\ &\mathbb{P}(\text{termination}) = 1 \quad \text{and} \quad \mathbb{E}[\text{cost}] = v. \end{aligned}$$

The first is existential reachability. The second asks for a persistent strategy that visits  $B$  infinitely often. The third is a statement about a measure on executions. A solver that forgets which question it is answering can return a perfectly valid certificate for the wrong proposition.

The proposed router therefore recognizes a *behavioral contract*, not just syntactic occurrences of recursion, a graph, or a matrix. An empty-stack query can enter the pushdown lane. A universally quantified infinite play under an existential policy can enter the game lane. A hitting-time or expected-cost claim can enter the probabilistic lane only after transition probabilities and terminal behavior have been preserved. Unsupported source constructs remain outside these fragments; the worker must not manufacture a finite model by silently erasing them.

## 1.3 What is already executable

The package uses the Python standard library only. All probabilities and numerical certificates use `Fractions.Fraction`; no floating-point solve is used. There are three search modules and one replay module. Replay imports neither search nor the algorithms used by the independent reference oracles. The input dataclasses and rational arithmetic runtime are shared, so this is algorithmic separation, not a proof of independent implementation correctness.

The payload of a successful Python replay is a *research result*. It is not a Lean expression, a theorem declaration, or a trusted receipt from the existing Forge boundary. In the proposed Lean implementation, a successful certificate check must still yield a proof of the corresponding object-level semantic theorem and be transported back to the original goal. These are explicitly separate milestones throughout the article.

## 2 Unbounded stacks: productive summaries and closed exclusions

**Evidence status.** Implemented and tested in Python, including the regular-target compiler. The suffix bridge, summary checker soundness, and source reifier have not been implemented or checked in Lean.

### 2.1 Input language and the question being decided

A normalized pushdown system is a tuple  $(Q, \Gamma, \Delta)$  with finite control set  $Q$ , finite stack alphabet  $\Gamma$ , and rules

$$(p, A) \longrightarrow (q, w), \quad |w| \leq 2.$$

The top of the stack is its *first* symbol. A rule induces

$$(p, A\beta) \longrightarrow (q, w\beta)$$

for every suffix  $\beta$ . There is no transition from an empty stack. The input also specifies an initial configuration  $(p_0, \alpha)$  and a set of final controls  $F$ . The basic query is

$$\exists f \in F : (p_0, \alpha) \longrightarrow^* (f, \varepsilon). \quad (1)$$

Finite control and alphabet do not make the configuration space finite. Rules may repeatedly increase the stack height. The worker does not impose a height, run-length, or recursion-depth bound.

The length-two restriction is part of the accepted input language, not an unchecked preprocessing convention. A future source adapter can normalize other transition syntax, but it must prove its own correspondence theorem. Arbitrary data-bearing frames, two stacks, higher-order stacks, and unbounded integer control variables are not accepted simply because a program happens to use recursion.

This is a specialization of the established saturation approach to pushdown reachability [7]. The specialization makes both the certificate and its source theorem especially small.

## 2.2 The finite object that summarizes an infinite configuration space

Write  $S(p, A, q)$  when the one-symbol stack can be removed:

$$S(p, A, q) \iff (p, A) \longrightarrow^* (q, \varepsilon).$$

There are only  $|Q|^2|\Gamma|$  possible triples. The essential feature of this relation is that it describes a transformation valid above *every* suffix, rather than a finite selection of suffixes.

**Lemma 2.1** (Suffix lifting). *If  $S(p, A, q)$ , then for every  $\beta$  there is a run  $(p, A\beta) \longrightarrow^* (q, \beta)$  that does not inspect or modify  $\beta$  before its final configuration.*

*Proof.* Take a run from the one-symbol configuration to the empty stack. Every configuration before its final one has a nonempty stack: after becoming empty it could make no further step. Append  $\beta$  to every stack in the run. Each step remains an instance of the same rule, because rules inspect only the top symbol and preserve the rest. The appended suffix is therefore untouched.  $\square$

**Remark 2.2** (Do not weaken the definition). An arbitrary run from  $(p, A\beta)$  to  $(q, \beta)$  might first expose, inspect, and later reconstruct  $\beta$ . Such a run is not by itself a suffix-parametric pop summary. The definition uses the one-symbol run, and the bridge proves protected suffix lifting. This distinction matters when translating recursive programs with stack-dependent behavior.

The summary relation is generated by three local rules:

$$\frac{(p, A) \rightarrow (q, \varepsilon) \in \Delta}{S(p, A, q)} \quad (\text{pop}), \quad (2)$$

$$\frac{(p, A) \rightarrow (r, B) \in \Delta \quad S(r, B, q)}{S(p, A, q)} \quad (\text{one child}), \quad (3)$$

$$\frac{(p, A) \rightarrow (r, BC) \in \Delta \quad S(r, B, m) \quad S(m, C, q)}{S(p, A, q)} \quad (\text{two children}). \quad (4)$$

These rules describe a least fixed point. A relation containing a circular justification is not automatically evidence of reachability.

**Theorem 2.3** (Exactness of pop-summary closure). *The least relation closed under (2)–(4) is precisely  $S$ . Consequently (1), for  $\alpha = A_1 \cdots A_\ell$ , holds exactly when there are controls*

$$q_0 = p_0, \quad q_\ell \in F, \quad q_i \in Q \ (0 \leq i \leq \ell) \quad \text{with} \quad S(q_{i-1}, A_i, q_i) \quad (1 \leq i \leq \ell),$$

*Only the last control is required to belong to  $F$ .*

*Proof.* For soundness, induct on a finite derivation. A pop is a real transition. A one-child rule is followed by the child's run. For a two-child rule, lift the first child's run over the suffix  $C$ , then run the second child. Thus every derived triple describes a genuine pop.

For completeness, induct on the number of transitions in a run removing a one-symbol stack. Inspect its first rule. A zero-symbol replacement finishes the run, because the empty stack cannot move. A one-symbol replacement leaves a strictly shorter pop run. A two-symbol replacement produces  $BC$ . Before  $C$  can be removed, the part above it must disappear; take the first moment when  $C$  is exposed. Up to this moment the machine cannot have inspected  $C$ . Removing this protected suffix gives a run that pops  $B$  to some control  $m$ . The remaining segment pops  $C$ . Both segments are shorter than the original run, so induction supplies both premises of (4).

For a longer initial word, decompose a run at the first exposure of each successive original suffix. This gives the controls  $q_i$ . Conversely, compose the summary runs using Lemma 2.1. For  $\ell = 0$ , the criterion is just  $p_0 \in F$ , correctly accepting a zero-step run.  $\square$

### 2.3 Search: indexed semi-naive saturation

The executable solver maintains a table of discovered triples and an agenda of new triples. A triple is inserted once. Alongside it, the solver records one rule identifier and the zero, one, or two earlier triples used to derive it.

Three indexes avoid rescanning the whole rule set. A unary rule is indexed by its required child head  $(r, B)$ . A binary rule is indexed both by its first child head  $(r, B)$  and by its second stack symbol  $C$ . A fourth index maps a summary head  $(m, C)$  to all currently known endpoints. When a new summary arrives, it can therefore act as either child of a binary rule. Both directions are necessary: the first child may be found after the second.

Listing 1: Agenda skeleton; the accompanying implementation includes both join directions.

```
insert all pop-rule summaries, with zero children
while agenda is nonempty:
    take a fresh summary (r, B, m)
    for every unary rule requiring (r, B):
        insert its conclusion using this child
    for every binary rule requiring (r, B) as its first child:
        join with all known (m, C, q)
        insert conclusions with the two child identifiers
    for every binary rule whose second symbol is B:
        look up its first child ending at r
        when present, join it with the new (r, B, m)
query the completed relation along the exact initial stack
return a sliced productive DAG, or a closed exclusion relation
```

Let  $n = |Q|$ ,  $a = |\Gamma|$ ,  $r_i$  be the number of rules replacing the top symbol by  $i$  symbols, and  $M \leq n^2 a$  the number of summaries. The core indexed closure performs

$$O(M + r_0 + r_1 n + r_2 n^2)$$

insertions, joins, and table operations under constant-time lookup assumptions. The bound counts combinations of a rule, a middle control, and an endpoint; there are no unbounded configuration objects. Storage for the closure and provenance is  $O(M + |\Delta|)$  records. Deterministic sorting, dictionary implementation costs, and Python object overhead are additional.

Querying an initial stack of length  $\ell$  is a finite relational composition, with  $O(\ell n^2)$  endpoint tests in a simple representation. The prototype copies short root paths while doing this query; a production implementation should retain predecessor links instead, eliminating quadratic path-copying costs on long initial words. These implementation details do not affect the fixed-point termination proof.

## 2.4 Positive certificate: productivity, not just closure

A positive certificate contains a topologically ordered array of nodes. Node  $i$  records a triple  $(p, A, q)$ , an input rule identifier, and child identifiers strictly smaller than  $i$ . The checker obtains the rule from the separately supplied problem, never from a rule description inside the certificate. It checks that the rule's source matches  $(p, A)$ , that its replacement symbols match the children in order, and that the children's control endpoints form a chain ending at  $q$ .

A separate root list has exactly one node identifier per symbol in the actual initial stack. It must start at the actual initial control and end in the actual final set. The checker rejects duplicate summary triples, forward references, self-references, illegal identifiers, omitted root symbols, and mismatched control chains. It does not accept a certificate-supplied claim that the target has been reached.

**Proposition 2.4** (Positive replay soundness). *If this checker accepts a positive certificate for a valid input, then (1) holds.*

*Proof.* Induct on the node index. Every child has already been justified, and the local rule check is exactly one of (2)–(4). Each root node therefore denotes a valid summary. Apply the root-chain part of Theorem 2.3.  $\square$

This certificate is naturally represented by shared Lean lemmas of type  $\text{Pop } p \ A \ q$ , not by a fully expanded list of transitions. A final bridge can derive existential reachability from those lemmas without normalizing an enormous concrete trace. The search chooses one derivation per triple and slices away all nodes not needed by the selected root chain.

For debugging, the Python checker also computes the length of the represented run:

$$L_i = 1 + \sum_{j \in \text{children}(i)} L_j.$$

Repeated child identifiers contribute repeatedly to the length, even though their proofs are stored once. The local replay uses  $O(K + \ell)$  node/reference checks for  $K$  stored nodes, in addition to input validation. The arithmetic cost of the possibly large  $L_i$  must not be hidden in a unit-cost claim. A Lean checker need not compute these diagnostic lengths at all.

**Example 2.5** (An exponentially long witness in a linear-size DAG). With one control, let

$$A_0 \rightarrow \varepsilon, \quad A_i \rightarrow A_{i-1}A_{i-1} \quad (1 \leq i \leq d).$$

Starting from  $A_d$ , the only run to empty has length  $L_d = 2^{d+1} - 1$ . There is one summary node per  $A_i$ ; each nonbase node points twice to its predecessor. At  $d = 30$ , replay checks 31 nodes and reports 2, 147, 483, 647 represented steps. The experiments expand the small cases  $d \leq 10$  into concrete traces, but do not execute the billion-step run. The large length is derived from the certificate recurrence.

## 2.5 Negative certificate: a closed over-approximation

A negative certificate supplies a finite relation  $C \subseteq Q \times \Gamma \times Q$ . The checker independently enumerates the consequences of every input rule and verifies that  $C$  contains the pop bases and is closed under both composition rules. It then runs the exact initial stack query against  $C$  and requires that no final control is admitted.

**Proposition 2.6** (Negative replay soundness). *If the exclusion checker accepts  $C$ , then no run in (1) exists.*

*Proof.* By induction on productive summary derivations, every actual summary belongs to  $C$ . If a target run existed, its decomposition from Theorem 2.3 would supply a chain in  $C$  ending in a final control, contradicting the checker's last test.  $\square$

Here a larger relation is safe: it can prevent a negative certificate from being accepted, but cannot create a false nonreachability proof. The polarity is reversed for a positive claim. For example, the rule  $A \rightarrow A$  does not pop anything; nevertheless the relation containing  $(p, A, p)$  is closed under that rule. It would be an unsound positive witness, whereas the empty relation is a valid exclusion for the pop query. A dedicated unit test submits precisely the self-supporting positive node and verifies its rejection.

The negative checker does not call the agenda solver. Its deliberately simple nested-loop audit costs  $O(r_0 + r_1n + r_2n^2 + \ell n^2 + M)$  relation operations. It need not check that  $C$  is the *smallest* closed relation. The solver emits the least relation because that is enough to decide every query in the accepted fragment.

**Example 2.7** (Safety despite unbounded reachable stacks). Let the initial control be  $p$ , with rules  $pA \rightarrow pAA$  and  $pA \rightarrow p\varepsilon$ , and ask whether the empty stack can be reached at a different control  $q$ . The reachable configurations include  $pA^k$  for arbitrarily large  $k$ . A finite enumeration of concrete configurations cannot cover them all. The one-triple relation  $C = \{(p, A, p)\}$  is nevertheless a closed exclusion certificate for the target. A direct search with a height cap reports truncation; the summary worker proves nonreachability. This is not a claim that all finite abstractions fail—the relation  $C$  is itself the useful finite abstraction.

## 2.6 Regular target languages, with an explicit bottom marker

Empty-stack reachability is not the most convenient source interface. A user may ask whether an error control can be reached with a stack matching a regular pattern. The shipped compiler accepts

a total DFA with states  $D$ , transition  $\delta : D \times \Gamma \rightarrow D$ , accepting set  $H$ , and one initial DFA state  $i(q)$  for each source control. Its target predicate is

$$\text{Target}(q, \beta) \iff \delta^*(i(q), \beta) \in H.$$

The DFA reads top to bottom. The original input's empty-stack final-control set is deliberately ignored by this interface: the supplied regular target replaces it, rather than being intersected with it silently.

The reduction appends a fresh bottom symbol  $\perp$  to the initial stack and adds fresh monitor controls  $\bar{d}$  and a final control *done*. The original rules cannot read or create  $\perp$ . For each original control  $q$  and symbol  $A \in \Gamma \cup \{\perp\}$ , add a one-way switch

$$(q, A) \rightarrow (\overline{i(q)}, A).$$

For  $A \in \Gamma$ , add

$$(\bar{d}, A) \rightarrow (\overline{\delta(\bar{d}, A)}, \varepsilon),$$

and for  $d \in H$  add  $(\bar{d}, \perp) \rightarrow (\text{done}, \varepsilon)$ . There is no return from a monitor control to an original control. The new target is empty stack at *done*.

**Theorem 2.8** (Regular-target reduction). *The compiled system reaches its empty-stack target exactly when the original system reaches a configuration satisfying Target.*

*Proof.* Given an original target run, append  $\perp$  to its stacks, take the switch, and consume the stack with the DFA monitor. Its final DFA state is accepting, so it can remove  $\perp$  and enter *done*.

Conversely, a compiled accepting run must enter the monitor, since only the monitor can reach *done*. Before its first switch it uses only original rules above the untouched bottom symbol. After that switch, its control is determined by the DFA scan. The bottom pop can occur only after the original stack has been consumed and the DFA has accepted it. The prefix before the switch therefore corresponds to an original run reaching the regular target. The bottom symbol also handles an already empty original stack, where no ordinary source rule would be available.  $\square$

With  $k = |D|$ , the reduction has  $n + k + 1$  controls,  $a + 1$  stack symbols, and  $|\Delta| + n(a + 1) + ka + |H|$  rules. The regression suite compares 100 compiled queries with a complete concrete oracle on non-growing original systems. It also distinguishes the words 01 and 10 to catch a reversed stack scan. These tests are evidence about the compiler implementation, not a formal proof of the reduction theorem in Lean.

## 2.7 How Forge would use the worker

An initial source adapter should accept a user-supplied pushdown model plus a proved operational correspondence, rather than infer arbitrary program semantics. Examples include a recursive interpreter with finitely many control locations and stack-frame tags, a context-free parser with a finite alphabet, or an error-state query for a recursive protocol. Once that adapter works, recognized definitions can be translated automatically using reusable source lemmas.

For a safety theorem, a forward simulation into the pushdown model can be enough: every source error execution must map to an abstract error execution. For a positive reachability theorem, an abstract witness must be concretizable; a one-way over-approximation cannot justify it. Section 5 spells out the corresponding proof obligations rather than treating a shared syntax tree as evidence that a translation is exact.

### 3 Persistent witnesses: Büchi strategies and their dual certificates

**Evidence status.** A finite total-arena Büchi solver and both certificate checkers are implemented in Python. Generalized acceptance and source-level strategy extraction are specified extensions, not executed features.

#### 3.1 A strategy is a witness with obligations over its future

Let  $G = (V, V_0, V_1, E, B)$  be a finite directed arena. The ownership sets partition  $V$ , every vertex has at least one successor, and  $B \subseteq V$  is the set of accepting vertices. Player 0 is the protagonist; player 1 is the antagonist. A play is an infinite sequence  $v_0v_1 \dots$  following  $E$ . Each owner chooses the successor when its vertex is visited.

The positive question at an initial vertex  $s$  is

$$\exists \sigma \forall \pi \text{ consistent with } \sigma \text{ from } s, \quad \forall N \in \mathbb{N} \exists n \geq N : v_n \in B. \quad (5)$$

The expression on the right is the Büchi condition, denoted  $\text{GFB}$ . A strategy may in general inspect the play history. On finite Büchi games, positional strategies, depending only on the current vertex, suffice for both players [8]. The shipped certificate language therefore contains a finite policy table, not a program with unbounded history.

A single path through  $B$  proves neither recurrence nor resistance to an opponent. A witness synthesized separately for each finite sample proves neither that the same policy works for all samples nor that it works forever. The policy itself must be the persistent witness, and its behavioral law must quantify over *all* permitted future choices.

The input validator rejects deadlocks. Whether a deadlock should lose, win, or be completed by a self-loop depends on the source semantics. Completing it without an explicit convention changes the meaning of (5). Likewise, any fairness restriction on an opponent must be represented as part of the objective or the model; no fairness is assumed here.

#### 3.2 Positive certificate: decrease until acceptance, then reset

A positive certificate consists of a region  $W \subseteq V$  containing  $s$ , a legal positional choice  $\sigma(v)$  at every  $v \in W \cap V_0$ , and a natural rank  $\rho : W \rightarrow \mathbb{N}$ . Define the allowed edges under this policy by

$$E_\sigma(v) = \begin{cases} \{\sigma(v)\}, & v \in V_0, \\ E(v), & v \in V_1. \end{cases}$$

The checker verifies

$$E_\sigma(v) \subseteq W \quad (v \in W), \quad (6)$$

$$\rho(v) > \rho(w) \quad (v \in W \setminus B, w \in E_\sigma(v)). \quad (7)$$

There is no decrease requirement when the *source* vertex belongs to  $B$ . At such a vertex the rank may rise again. Closure still applies there.

**Theorem 3.1** (Positive rank soundness). *An accepted positive certificate proves (5) with its supplied policy, extended arbitrarily outside  $W$ .*

*Proof.* Closure keeps every consistent play in  $W$ . Suppose some such play visits  $B$  only finitely often. Beyond its last accepting visit, every edge has a source outside  $B$ , so (7) gives an infinite strictly decreasing sequence of natural numbers. This is impossible. Equivalently, from any nonaccepting position, at most its current rank many transitions can occur before an accepting vertex is reached. Apply this argument after every finite prefix to obtain the quantifiers in (5).  $\square$

This is not ordinary termination. For the two-vertex cycle  $a \rightarrow b \rightarrow a$  with  $B = \{a\}$ , take  $\rho(a) = 0$  and  $\rho(b) = 1$ . The edge  $b \rightarrow a$  decreases;  $a \rightarrow b$  resets the rank. Every run is infinite and accepting. A globally strictly decreasing ranking would reject this correct recurrent behavior. This is the specific difference from Forge’s existing compilation of cyclic proof obligations to well-founded descent.

There is a useful bounded response property hidden in the same certificate. Let  $H = \max_{v \in W} \rho(v)$ . Starting at a nonaccepting vertex, acceptance occurs within  $H$  transitions. Starting at an accepting vertex, a later accepting vertex occurs within  $H + 1$  transitions. The certificate thus gives a coarse recurrence-gap bound as well as qualitative infinite recurrence. This bound concerns graph steps; a source adapter with variable-duration steps needs an additional cost or timing argument.

### 3.3 Negative certificate: bound all future accepting visits

A negative result must do more than exhibit one bad play. It must show that *no* protagonist strategy satisfies the objective. The dual witness is an antagonist strategy  $\tau$  that defeats every protagonist continuation.

Its certificate consists of  $L \subseteq V$  containing  $s$ , a legal choice  $\tau(v)$  at every  $v \in L \cap V_1$ , and  $\kappa : L \rightarrow \mathbb{N}$ . Now the allowed edges are

$$E_\tau(v) = \begin{cases} E(v), & v \in V_0, \\ \{\tau(v)\}, & v \in V_1. \end{cases}$$

The local checks are

$$E_\tau(v) \subseteq L, \tag{8}$$

$$\kappa(v) \geq \kappa(w) + \mathbf{1}_B(v) \quad (w \in E_\tau(v)). \tag{9}$$

Unlike the positive rank, this rank never increases. Each departure from an accepting vertex consumes at least one unit. Nonaccepting cycles of constant rank are allowed, and are usually the intended counterbehavior.

**Theorem 3.2** (Negative rank soundness). *An accepted negative certificate forces at most  $\kappa(s)$  accepting visits on every infinite play consistent with  $\tau$  from  $s$ . In particular, (5) is false.*

*Proof.* For every finite prefix of length  $N$ , summing (9) gives

$$\sum_{i=0}^{N-1} \mathbf{1}_B(v_i) \leq \kappa(v_0) - \kappa(v_N) \leq \kappa(s).$$

Closure makes every rank in this expression defined. The bound holds for every  $N$ , so infinitely many accepting visits are impossible. The certificate checks every protagonist edge, hence the conclusion does not depend on a particular protagonist policy.  $\square$

The distinction between a counterstrategy and a counterexample lasso is operationally important. A lasso can refute one fixed candidate policy. It cannot refute the existence of a different winning policy. The negative certificate leaves all player-0 choices available and checks them universally.

### 3.4 Search by repeated attractors

For a total subarena  $U$ , the player- $i$  attractor of  $T \subseteq U$  is the least set containing  $T$  and closed under the following additions: a player- $i$  vertex is added when it has a successor already in the set; a vertex owned by the other player is added when all its successors in  $U$  are already in the set. A layered construction records the round of addition and a rank-decreasing choice at the controlled vertices.

The solver uses the classical repeated-attractor scheme discussed in [8], not that paper’s faster specialized algorithm.

Listing 2: The implemented Buechi search, in semantic pseudocode.

```

U := all vertices
while U is nonempty:
  A := Attr_0(B intersect U) inside U
  D := U minus A
  if D is empty:
    return U as the winning region
  Z := Attr_1(D) inside U
  record player-1 choices that reach D and then avoid B
  remove Z from U
return the empty winning region

```

The maintained subarena is total: every remaining player-0 vertex has some remaining successor, and every successor of a remaining player-1 vertex remains. Thus the attractor computation never gains a spurious victory from an empty successor set. This invariant also explains why the final attractor policy remains winning in the original arena rather than only in an induced graph with some inconvenient opponent edges removed.

When  $D$  is nonempty, no vertex of  $D$  is accepting. At a player-0 vertex of  $D$ , every successor that remains in  $U$  stays in  $D$ ; at a player-1 vertex there is a successor in  $D$ . The antagonist can therefore reach  $D$  from  $Z$  and avoid  $B$  thereafter, unless the protagonist exits to an earlier removed region. That earlier region is already losing. Layer numbers can decrease only finitely often, so the assembled antagonist policy is co-Büchi winning. At least one vertex is removed in each unsuccessful outer iteration.

When the final  $U$  equals its attractor, the attractor levels supply the positive rank. At nonaccepting player-0 vertices the recorded move descends; at nonaccepting player-1 vertices all moves descend. At accepting player-0 vertices, choose any successor in  $U$ . No descent is required there.

The negative certificate intentionally does not expose the search’s removal layers as its authoritative argument. After fixing the assembled antagonist policy on the losing region, the implementation solves the local longest-weight problem

$$\kappa(v) = \max_{w \in E_\tau(v)} (\mathbf{1}_B(v) + \kappa(w)) \quad (10)$$

by monotone iteration from zero. An accepting vertex cannot occur on a cycle of this fixed-policy graph, or the policy would not force co-Büchi. Cycles therefore have weight zero. Removing repeated

zero-weight cycles from a walk shows that its total accepting-departure weight has a finite maximum, and the iteration stabilizes. The replay checker verifies only (8)–(9), not this optimization or the attractor computations that preceded it.

### 3.5 Completeness and computational costs

**Proposition 3.3** (Completeness of the finite rank languages). *Every winning initial vertex in a finite total Büchi game has a positive certificate of the preceding form; every losing initial vertex has a negative certificate.*

*Proof.* Choose positional winning strategies, whose existence follows from finite-game memoryless determinacy. Under a winning protagonist policy, a reachable cycle entirely outside  $B$  would be a losing play. The subgraph on nonaccepting vertices is therefore acyclic. Give an accepting vertex rank zero and a nonaccepting vertex the maximum number of transitions needed to reach the first accepting vertex in this policy graph. These ranks satisfy (7).

Under a winning antagonist policy, a reachable cycle containing an accepting vertex would allow infinitely many accepting visits. No such cycle exists. Give each vertex the maximum accepting-departure count of a finite walk from it. Nonaccepting cycles can be erased without reducing this count; the maximum is finite and satisfies (9). Restrict either construction to the corresponding strategy-closed region.  $\square$

The implemented attractor uses repeated scans for transparency. With  $n$  vertices and  $m$  edges, one attractor may require  $O(nm)$  work and there may be  $O(n)$  outer iterations, giving a coarse  $O(n^2m)$  bound for this Python version. It would be incorrect to label the implementation  $O(nm)$  merely because that bound is standard for the classical algorithm with better data structures.

The direct production improvement is a predecessor queue with a counter of not-yet-attracted successors for each opponent-owned vertex. Each edge is then processed once per attractor, yielding  $O(nm)$  for the outer algorithm. The negative longest-weight iteration also admits an SCC-condensation dynamic program. Neither optimization is required for soundness, and neither is claimed to have been measured here. The rank checker itself costs  $O(n + m)$  local tests, apart from integer bit costs and input validation.

### 3.6 Examples that separate the semantics

In the arena with edges  $s \rightarrow s$ ,  $s \rightarrow b$ ,  $b \rightarrow b$  and  $B = \{b\}$ , player 0 wins from  $s$  when it owns  $s$ : choose  $b$ . Change only the owner of  $s$  to player 1 and the antagonist can remain at  $s$  forever. The edge graph has not changed, but the correct verdict has. Tests check both outcomes and also reject the old policy after the ownership mutation.

In the arena  $a \rightarrow b$ ,  $b \rightarrow b$ , with  $B = \{a\}$  and start  $a$ , the target is visited immediately. The Büchi objective is still false: there is only one accepting visit. This is a regression against substituting ordinary reachability for a temporal specification.

A final trap is to omit an inconvenient opponent edge during replay. The positive checker reads all successors of every player-1 vertex from the original problem. A certificate cannot shrink this list. Similarly, the negative checker cannot remove a player-0 choice. These are semantic checks, not optional certificate-format hygiene.

### 3.7 A concrete next extension: several recurring obligations

For a conjunction  $\bigwedge_{j < k} \text{GF} B_j$ , a finite monitor can remember the next acceptance set that must be visited. This is a specified extension, not part of the shipped solver API. Use product states  $(v, j)$ , with  $j \in \{0, \dots, k-1\}$ , preserve the owner of  $v$ , and update on each departure from  $v$ :

$$j' = \begin{cases} (j+1) \bmod k, & v \in B_j, \\ j, & v \notin B_j. \end{cases}$$

Let the product accepting set consist of  $(v, k-1)$  with  $v \in B_{k-1}$ . A visit to this set completes a full round. Infinitely many completed rounds are equivalent to infinitely many visits to every  $B_j$ ; both directions follow by following the next-required index. A positional product strategy becomes a source strategy with  $k$  memory states.

Crucially, “the monitor index is zero” is *not* an adequate accepting condition: an execution may remain at index zero forever without satisfying  $B_0$ . The acceptance event must mean completion of a round. A future implementation should test a system stuck before its first required event and a system whose successful policy must alternate between two obligations. This extension remains finite and precise; it does not silently claim support for arbitrary fairness assumptions, partial observation, or general temporal logic.

## 4 Exact probabilistic termination and values

**Evidence status.** Finite rational-chain search, exact arithmetic replay, and closed-class counter-certificates are implemented in Python. The probability-space bridge and its Lean formalization remain proposed.

### 4.1 The input semantics must include probability mass

Let  $S$  be a finite nonempty state set and let  $P \in \mathbb{Q}^{S \times S}$  satisfy

$$P_{st} \geq 0, \quad \sum_t P_{st} = 1.$$

The input specifies an initial state  $s_0$ , a terminal set  $T \subseteq S$ , a nonnegative rational cost  $c(s)$  per nonterminal visit, and a terminal payoff  $g(t) \in [0, 1] \cap \mathbb{Q}$ . Every terminal state is absorbing; its cost is zero. The implementation stores  $g(s) = 0$  off  $T$ .

Write  $X_n$  for the Markov chain and  $\tau = \inf\{n \geq 0 : X_n \in T\}$ . The positive output means

$$\mathbb{P}_{s_0}(\tau < \infty) = 1, \quad h(s_0) = \mathbb{E}_{s_0} \tau, \quad v(s_0) = \mathbb{E}_{s_0} \sum_{k=0}^{\tau-1} c(X_k), \quad u(s_0) = \mathbb{E}_{s_0} g(X_\tau). \quad (11)$$

If  $g$  is the indicator of a successful terminal subset,  $u$  is the success probability. With arbitrary fractional terminal payoffs it is an expected payoff, not automatically the probability of a specified event. The returned field is therefore named `expected_terminal_payoff`; a regression test uses payoff 1/3 to preserve this distinction.

The validator rejects floating probabilities, negative entries, nonunit row sums, nonabsorbing terminals, and nonzero terminal costs. There is no tolerance parameter and no automatic normalization.

These checks are part of the source semantics. Multiplying a stochastic row by a normalizing factor after export would change the problem unless that normalization were itself justified.

The worker's complete decision question is almost-sure absorption in  $T$ . Exact costs and terminal values accompany positive answers. On a negative answer it supplies a certified positive lower bound for nontermination, not necessarily the exact nontermination probability or an exact total cost. This scope matters when some nonterminating behavior has zero running cost.

## 4.2 Why a harmonic equation is insufficient

A tempting certificate for a probability is a vector  $u$  satisfying  $u = Pu$  with the intended boundary values. This is insufficient without a condition that makes the boundary reachable in the appropriate probabilistic sense. For the one-state nonterminal self-loop  $P = [1]$ , every scalar  $u$  satisfies  $u = Pu$ . No such scalar by itself proves that the process ever reaches a terminal state; there is no terminal state at all.

Likewise, an exact solution of a floating-point-derived linear system is exact only for the rounded model. A weighted-word identity concerns products along finite words. Neither kind of evidence establishes that an infinite-horizon limit has the claimed boundary values. The missing ingredient is a *properness witness*: a finite nonnegative potential that pays at least one unit for every nonterminal step.

## 4.3 The certificate and its local checker

The certificate supplies a set  $R \subseteq S$  containing  $s_0$ , closed under all positive-probability successors, and three rational vectors  $h, v, u$  on  $R$ . The support need not be the smallest reachable set; it only has to be forward closed. Outside  $R$ , the serialized vectors contain no values.

The checker verifies  $h, v \geq 0$ ,  $0 \leq u \leq 1$ , the terminal conditions

$$h(t) = v(t) = 0, \quad u(t) = g(t) \quad (t \in R \cap T),$$

and, for every  $s \in R \setminus T$ ,

$$h(s) = 1 + \sum_{t \in R} P_{st} h(t), \quad (12)$$

$$v(s) = c(s) + \sum_{t \in R} P_{st} v(t), \quad (13)$$

$$u(s) = \sum_{t \in R} P_{st} u(t). \quad (14)$$

All equalities are exact rational identities. The checker performs matrix–vector products; it does not call Gaussian elimination, compute strongly connected components, or trust a solver-supplied residual norm.

**Theorem 4.1** (Properness and exact values). *If the positive checker accepts these data, then (11) holds. Moreover the same claims hold from every state in  $R$  with that state's certificate values.*

*Proof.* Forward closure ensures that a chain started in  $R$  stays there. Extend the nonterminal indicator by zero on  $T$ . Iterating (12) for a finite number  $N$  of steps gives the exact identity

$$h(s) = \sum_{k=0}^{N-1} \mathbb{P}_s(\tau > k) + \mathbb{E}_s[h(X_N)]. \quad (15)$$

The sum is  $\mathbb{E}_s[\min(\tau, N)]$ . Its terms and the residual are nonnegative. Letting  $N$  increase shows  $\mathbb{E}_s\tau \leq h(s) < \infty$ , and hence  $\mathbb{P}_s(\tau = \infty) = 0$ . One can avoid this first limit by observing directly that

$$(N+1)\mathbb{P}_s(\tau > N) \leq \sum_{k=0}^N \mathbb{P}_s(\tau > k) \leq h(s).$$

Thus the survival probability tends to zero.

Since  $R$  is finite,  $H = \max_{t \in R} h(t)$  is finite. The residual in (15) is at most  $H\mathbb{P}_s(\tau > N)$ , because  $h$  vanishes on terminals. It tends to zero. Therefore the finite sums converge exactly to  $h(s)$ , proving the expected-time equality rather than only an upper bound.

Similarly, iterating (13) yields

$$v(s) = \mathbb{E}_s \sum_{k=0}^{N-1} c(X_k) + \mathbb{E}_s[v(X_N)].$$

The residual is at most  $V\mathbb{P}_s(\tau > N)$ , where  $V = \max_{t \in R} v(t)$ . Terminal costs vanish and costs are nonnegative, so monotone convergence identifies the limit with the total pretermination cost. Its value is  $v(s)$ .

Finally, (14) also holds at terminals, because they are absorbing. Thus  $u(s) = \mathbb{E}_s[u(X_N)]$  for every  $N$ . On  $\{\tau \leq N\}$  this random variable equals  $g(X_\tau)$ ; on its complement it lies in  $[0, 1]$ . The complement's probability tends to zero, giving  $u(s) = \mathbb{E}_s[g(X_\tau)]$ . No assertion that an arbitrary harmonic solution is unique has been used without first proving properness.  $\square$

#### 4.4 A finite-horizon proof interface with explicit error bounds

The preceding proof offers a particularly small first Lean target. Let  $\mu_N$  be the exact finite distribution after  $N$  steps, defined recursively by  $\mu_0 = \delta_s$  and  $\mu_{N+1} = \mu_N P$ . Put

$$t_N = \sum_{x \notin T} \mu_N(x), \quad C_N = \sum_{k=0}^{N-1} \sum_x \mu_k(x) c(x), \quad G_N = \sum_{x \in T} \mu_N(x) g(x).$$

The checker theorem can first establish, entirely through finite sums and induction on  $N$ ,

$$t_N \leq \frac{h(s)}{N+1}, \tag{16}$$

$$0 \leq h(s) - \sum_{k=0}^{N-1} t_k \leq \frac{Hh(s)}{N+1}, \tag{17}$$

$$0 \leq v(s) - C_N \leq \frac{Vh(s)}{N+1}, \tag{18}$$

$$0 \leq u(s) - G_N \leq \frac{h(s)}{N+1}. \tag{19}$$

These are rational inequalities for each natural  $N$ , not numerical experiments. Terminal absorption makes  $t_N$  nonincreasing; the exact finite identities then give the bounds as in the proof. In particular,  $u(s) - G_N = \sum_{x \notin T} \mu_N(x) u(x)$ , so its sign and bound are explicit.

A formalization can prove these finite-sum identities before introducing an infinite path measure. It can then identify  $t_N$  with the probability of a hitting-time event using the standard probability

semantics, or use the sequences directly for a source language whose denotation is given as the limit of finite execution distributions. The latter interface still needs a source theorem; it must not call a finite-vector recurrence an operational probability statement without that link.

The bounds are conservative. Geometric examples have exponentially decaying tails, whereas (16) is only inverse-linear. Its advantages are that it follows from the same small certificate and makes every limiting step explicit. A later worker can certify geometric drift or multi-step contraction for sharper rates without changing the basic absorption checker.

#### 4.5 Search: graph structure first, rational solving second

The algorithm first computes the states reachable from  $s_0$  in the support graph, whose edges are exactly the entries  $P_{st} > 0$ . It decomposes this graph into strongly connected components and looks for a reachable bottom component containing no terminal state. If one exists, search returns the negative certificate below. Unreachable traps are irrelevant to this query and are not allowed to spoil a positive answer.

Otherwise let  $U = R \setminus T$ , let  $Q = P_{U,U}$ , and form

$$A = I - Q, \quad b_h = \mathbf{1}, \quad b_v = c|_U, \quad b_u(s) = \sum_{t \in T} P_{st}g(t).$$

The search solves the three right-hand sides simultaneously:

$$A [h \ v \ u] = [b_h \ b_v \ b_u]. \quad (20)$$

The prototype uses exact Gauss–Jordan elimination with rational pivots. It sets terminal values from the original input and returns the full vectors on  $R$ . The checker validates their signs and all original-model equations afresh.

Why is  $A$  invertible in the positive case? Every reachable nonterminal state has a support path to a terminal; otherwise its reachable subgraph would contain a nonterminal bottom component. Choose one such path from each state. Finiteness gives a common length bound  $m$  and a positive minimum path probability  $\delta$ . Conditional on not having terminated, the chain has probability at least  $\delta$  to terminate within the next  $m$  steps. Hence

$$\mathbb{P}_s(\tau > km) \leq (1 - \delta)^k.$$

The transient matrix powers tend to zero, the series  $\sum_{k \geq 0} Q^k$  converges, and it is the inverse of  $I - Q$ . This also shows that absorption in a finite chain entails finite expected absorption time. The corresponding assertion is not automatic for infinite-state chains.

A dense solve has  $O(|U|^3)$  rational field-operation cost. Exact bit complexity can be much larger because intermediate numerators and denominators grow. The graph phase is linear in the explicit support graph, although the shipped matrix representation also scans zero entries. Replay uses  $O(|R|^2)$  exact arithmetic operations in the dense representation and never factors or inverts a matrix.

For larger instances, `fmpq_mat_solve` in FLINT is a concrete rational linear-algebra backend [14]; modular or fraction-free methods can replace the prototype solve without enlarging the checker. Storm is another candidate search backend for richer probabilistic models [15]. Its model export must preserve exact probabilities. In particular, Storm’s explicit transition-file interface is documented as using floating input and not supporting the same exact mode as suitable symbolic-model inputs. Merely passing `-exact` through a wrapper is not a proof that the original probability model was retained.

#### 4.6 Negative certificate: a reachable closed nonterminal class

A negative certificate is a nonempty set  $C \subseteq S \setminus T$  and a finite path  $s_0, s_1, \dots, s_k$  ending in  $C$ . The checker verifies

$$P_{s_i s_{i+1}} > 0 \quad (i < k), \quad P_{xy} > 0 \implies y \in C \quad (x \in C). \quad (21)$$

It returns the exact rational

$$\delta = \prod_{i=0}^{k-1} P_{s_i s_{i+1}} > 0.$$

For a path of length zero, this product is one. The checker need not verify that  $C$  is strongly connected or minimal; closedness and nonterminality are sufficient. Those stronger properties help the search find a small object, but are not obligations of the certificate.

**Theorem 4.2** (Closed-class refutation). *An accepted negative certificate proves  $\mathbb{P}_{s_0}(\tau = \infty) \geq \delta > 0$ . Thus absorption is not almost sure.*

*Proof.* The specified finite path has probability  $\delta$ . After it enters  $C$ , the row-sum and closure conditions imply that all subsequent transitions stay in  $C$  with probability one. Since  $C$  contains no terminal, the chain never terminates on that event. A terminal cannot have appeared earlier on the path: it is absorbing and could not be left to reach  $C$ .  $\square$

Every negative instance of the finite-chain almost-sure question has such a certificate: some reachable bottom component is nonterminal. Together with Theorem 4.1, this makes the search a decision procedure on the stated finite rational fragment, subject only to implementation resources. The checker handles an intentionally simpler object than the SCC algorithm used by search.

The negative verdict alone does not imply that total expected *cost* is infinite: a nonterminal closed class can have zero costs. It does imply that expected hitting time is infinite, because  $\tau = \infty$  has positive probability. A source-level report must keep these two quantities separate.

#### 4.7 Worked examples and adversarial distinctions

For a two-state chain with a retry state  $r$  and successful terminal  $t$ , let

$$P = \begin{pmatrix} 3/4 & 1/4 \\ 0 & 1 \end{pmatrix}, \quad c(r) = 3/2, \quad g(t) = 1.$$

The certificate is

$$h = (4, 0), \quad v = (6, 0), \quad u = (1, 1).$$

For example  $4 = 1 + (3/4)4$  and  $6 = 3/2 + (3/4)6$ . The checker therefore proves almost-sure termination, expected time four, expected cost six, and terminal success probability one. The infinite sequence that remains at  $r$  forever is still a syntactically possible path, but it has probability zero. Replacing the probabilistic choice by an antagonist that can choose the self-loop makes universal eventual termination false. This example is shared with the game demonstration precisely to expose the difference in quantifiers.

Now let a start state go to a successful terminal with probability  $2/3$  and to a nonterminal absorbing state with probability  $1/3$ . The singleton trap and the one-edge path to it certify nontermination probability at least  $1/3$ . No linear solve is needed. Changing that edge to probability zero must invalidate the path certificate, even if the same vertices remain in the serialized model.

A very small positive exit probability can also have a very large expected waiting time. For exit probability  $1/2^{40}$ , the exact expected time is  $2^{40}$ . A tolerance that rounds the exit probability to zero changes an almost-surely terminating chain into a nonterminating one. Conversely, rounding a truly zero edge upward can erase a trap. The exact input and exact residual discipline address this semantic discontinuity; floating residual size alone does not.

## 4.8 Extensions that require additional certificates

For a Markov decision process, graph SCCs must be replaced by the relevant end-component analysis, and a policy becomes part of the certificate. A properness equation for *one* policy proves neither almost-sure termination under every policy nor an optimum among policies. One concrete sufficient certificate for a universal time bound is a potential satisfying

$$h(s) \geq 1 + \sum_t P(s, a, t)h(t)$$

for every enabled action at every nonterminal state. Exact optimal values need Bellman inequalities for all alternatives and an attaining policy, together with the appropriate properness conditions. These are specific next steps, not claims that the present chain solver handles nondeterministic scheduling.

Similarly, exact reachability probabilities in a chain that is not almost surely absorbed in the original  $T$  can be reduced by making nonterminal bottom classes explicit failure terminals. That reduction preserves the reachability event but not the original expected hitting time or total running cost. A future adapter must state which quantity it preserves. The current negative result deliberately returns only its proved lower bound rather than making an unchecked reduction or overstating the available value information.

## 5 The source bridges are the integration, not a footnote

### 5.1 A checked finite model is not yet a proof of the original goal

Forge’s existing design already separates source identity, search guidance, certificate acceptance, and theorem authority [4]. Those mechanisms should be reused, not redesigned here. The new obligation is more specific: each worker has a different *semantic direction* and therefore needs a different translation theorem.

The initial implementation should take an explicit model and a proved bridge. Only after this route works should a reifier infer the model from arbitrary Lean definitions. That order gives an end-to-end theorem sooner and prevents an attractive front-end demo from concealing an unproved encoding. A useful worker result is thus conceptually a dependent pair

$$(\text{model certificate, proof of the model proposition}),$$

which is then consumed by the source bridge. The model proposition is not allowed to change when proof search is inconvenient.

### 5.2 Reachability: forward simulation and witness lifting differ

Let  $\mathcal{C}$  be a source transition system, let  $\mathcal{A}$  be the pushdown model, and let  $\mathcal{R}(c, a)$  relate source and abstract configurations. To transfer an abstract safety result, it is sufficient to prove an initial

relation, preservation of the relation along every source step by a finite abstract run, and a target mapping

$$\text{Bad}_C(c) \wedge \mathcal{R}(c, a) \implies \text{Target}_A(a).$$

An abstract nonreachability theorem then rules out a source error. Abstract behavior may be an over-approximation; extra abstract paths weaken the ability to prove safety but do not invalidate a proved safety result.

A positive abstract run is different. It requires a lifting theorem that constructs a related source run, or an exact operational correspondence. A spurious abstract path cannot establish a source witness. A practical first adapter should use an equivalence of the two reachability propositions, with both directions proved once for its recognized syntax.

For regular stack targets, the target DFA is also part of the bridge. The compiler theorem must connect *the original stack predicate* to the DFA language, then to the monitor phase, then to the empty-stack query. A proof that the compiled automaton empties its stack does not by itself identify what that stack meant in the source program. Top-first order and the fresh bottom symbol must occur in the theorem, not only in a comment in the exporter.

### 5.3 Games need alternating, ownership-aware refinement

A forward simulation of unlabelled graphs is not sufficient for strategy synthesis. The source may distinguish actions the program controls from outcomes controlled by an environment. At a source state  $x$ , an abstract protagonist move must have a legal *uniformly realizable* implementation; every possible environment outcome of that implementation must be accounted for.

A precise first interface uses a turn-based source arena with an abstraction  $\alpha : X \rightarrow V$ . For a positive policy  $\sigma$  on  $W$ , require a source invariant  $I$ , the initial condition  $I(x_0)$  and  $\alpha(x_0) \in W$ , and the following obligations:

1. Ownership is preserved. At a protagonist state satisfying  $I$ , a checked realizer chooses a legal source successor mapping to  $\sigma(\alpha(x))$  and preserving  $I$ .
2. At an antagonist state satisfying  $I$ , *every* legal source successor preserves  $I$  and maps to a successor of  $\alpha(x)$  in the original abstract arena.
3. Abstract acceptance implies source acceptance:  $I(x) \wedge \alpha(x) \in B \implies B_{\text{src}}(x)$ . Source plays are infinite under the relevant strategy, or the source's treatment of termination is explicitly reflected in the arena.

Under these conditions the abstract policy lifts to a source policy, every consistent source play maps stepwise to a consistent abstract play, and Theorem 3.1 transfers. The realizer is a typed function whose correctness is quantified over all source states satisfying  $I$ , not a lookup that was tested on one representative of each abstract block.

When a source action has several possible outcomes, selecting the action is not the same as selecting an outcome. A concrete encoding inserts an intermediate antagonist-owned action vertex: the protagonist chooses the vertex for the action, then the antagonist chooses its outcome. Labels on these intermediate vertices and the two-step-to-one-step correspondence must be fixed in the bridge. Simply turning every possible action outcome into a protagonist edge would overstate the program's power.

A negative game certificate reverses the controlled side. To conclude that *no source strategy* succeeds, the abstract antagonist policy must lift, all protagonist choices must be represented, and source

accepting visits must imply the relevant abstract accepting visits. The positive label implication alone is not enough for this direction. The implementation should expose separate positive and negative bridge types rather than a single vaguely named “simulation” field.

#### 5.4 Stuttering can destroy liveness even when it preserves safety

Suppose the abstract system takes  $a \rightarrow b$ , with  $b$  accepting forever. Suppose the source can perform infinitely many internal steps that all abstract to  $a$ . A weak simulation that ignores internal steps may still relate every finite source prefix to the abstract initial prefix. It does not establish that the source ever reaches an accepting state.

There are two suitable first implementations. The simpler one requires each source step to map to an abstract step, as above. The more flexible one allows stuttering only when supplied with a well-founded rank that strictly decreases during internal steps and is reset only when an abstract step occurs. An infinite source play must then contain infinitely many abstract steps. If acceptance transfer is also aligned with those steps, abstract Büchi recurrence implies source recurrence. This use of a stuttering rank complements the new reset-at-acceptance rank; the two ranks certify different things.

Quantitative recurrence-gap bounds do not transfer through this more flexible bridge without a bound on the number or duration of internal steps. Qualitative liveness and bounded response time should remain distinct result types.

#### 5.5 Probability requires exact pushforward, not support simulation

Let a source probabilistic program have transition distribution  $K(x)$  and let  $\alpha : X \rightarrow S$  map source states to the finite chain. A sufficient exact quotient condition is

$$\alpha_* K(x) = P_{\alpha(x)}, \quad \text{for every source state satisfying the invariant.} \quad (22)$$

Also require invariant preservation with probability one, the initial-state mapping, and exact correspondence of terminals, running costs, and terminal payoffs:

$$\begin{aligned} x \in T_{\text{src}} &\iff \alpha(x) \in T, \\ c_{\text{src}}(x) &= c(\alpha(x)), \\ g_{\text{src}}(x) &= g(\alpha(x)) \quad (x \in T_{\text{src}}). \end{aligned}$$

Induction on finite horizons then equates the abstract distributions with the pushforwards of the source distributions. The finite-horizon identities and limits from Section 4 transfer to the source.

This is a strong but useful condition: the source state space can be infinite provided the finite quotient is exact. It is not enough that source and abstract transitions have the same possible endpoints. Two source states with exit probabilities  $1/4$  and  $1/2$  cannot generally be merged into one abstract state with a single transition row. An average row has no justification merely because it falls between the two probabilities. Formula (22) asks for mass equality for every abstract block, not an empirical average.

Mathlib’s probability mass functions and probability kernels provide the appropriate semantic objects [11, 12]. For discrete source programs, the intended equation is a distribution-map equality; for a measure-theoretic kernel it is the corresponding pushforward equality. The finite rational matrix must be linked to that object with proofs of nonnegativity and unit mass. No claim is

made that importing the probability library supplies this application-specific quotient theorem automatically.

## 5.6 What to borrow from Leant and Djex, without repeating their work

Leant and Djex already preserve source-owned typed candidates, universe and binder selections, and scoped provider evidence, and Leant re-elaborates candidate terms [5, 6]. The new use is to ask their synthesis layer for the *realizer* or *policy implementation* demanded by an accepted behavioral certificate.

For example, the game solver can propose a finite abstract policy while the typed synthesizer builds a source action-selection function from available providers. Forge then proves the universal realization and refinement conditions. A syntactically different function of the same type is not an acceptable substitute if its behavior differs. Conversely, a well-typed source function does not receive the temporal theorem merely because its finite observations agree with the policy on a sample.

The semantic proof belongs to the exact retained candidate, with its lexical providers and dictionaries. This is an application of the neighboring projects' existing ownership discipline, not a new discipline claimed by this article. It also does not resolve their separate open Church-encoding or higher-universe synthesis cases. The proposed integration adds temporal contracts to selected typed candidates; it does not assert a complete higher-order program synthesizer.

## 6 A concrete Lean implementation plan

### 6.1 New modules, not a replacement for the existing controller

The proposed namespace is `Forge.Horizon`; it is a design namespace, not an existing repository API. The existing `forge` controller should route an accepted semantic contract to one of these modules and receive an ordinary proved consequence. Local equality and arithmetic reasoning can remain with `grind`; this proposal does not require rewriting its saturation engine [9].

| Proposed module                             | Concrete responsibility                                                                                   |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>Horizon/Pushdown/Semantics</code>     | Define configurations, top-first rules, finite runs, and the suffix-lifting theorem.                      |
| <code>Horizon/Pushdown/Check</code>         | Check productive DAGs and closed exclusions; prove the two semantic soundness theorems.                   |
| <code>Horizon/Pushdown/RegularTarget</code> | Define the one-way DFA monitor and prove both directions of the target reduction.                         |
| <code>Horizon/Buchi/Semantics</code>        | Define total turn-based arenas, positional policies, consistent infinite plays, and recurrent acceptance. |
| <code>Horizon/Buchi/Check</code>            | Check winning and losing ranks; prove recurrence and bounded accepting-visit count.                       |
| <code>Horizon/Markov/Finite</code>          | Define rational stochastic matrices and exact finite execution distributions.                             |
| <code>Horizon/Markov/Check</code>           | Check properness, cost and payoff residuals, and closed-class path certificates.                          |
| <code>Horizon/Markov/Probability</code>     | Relate finite distributions and their limits to PMFs, kernels, and hitting-time semantics.                |

| Proposed module | Concrete responsibility                                                                          |
|-----------------|--------------------------------------------------------------------------------------------------|
| Horizon/Bridge  | Keep reachability, positive-game, negative-game and probability refinement obligations distinct. |
| Horizon/Elab    | Recognize supported goals, quote data, invoke search, replay, and apply a source bridge.         |

The corresponding Python implementation already separates `pushdown.py`, `games.py`, and `markov.py` from `checkers.py`. Only the latter needs a direct semantic port for the first Lean checker milestone; the Python searches may remain untrusted helpers. The complete standard-library source is included, rather than a dependency on an unavailable external service.

## 6.2 The exact theorems to implement first

Here is specification notation for the proposed APIs. The names are new, and this listing is not claimed to be compiled Lean syntax:

Listing 3: Required soundness contracts, with the original model supplied independently.

```

checkPopDAG p cert = true
  -> ReachEmpty p p.start p.stack p.finals

checkPopExclusion p cert = true
  -> not (ReachEmpty p p.start p.stack p.finals)

checkBuchiWin g start cert = true
  -> exists policy,
    forall play, ConsistentPlay g policy start play
      -> forall N, exists n >= N, Accepting g (play n)

checkBuchiLose g start cert = true
  -> exists counterpolicy,
    forall play, ConsistentCounterplay g counterpolicy start play
      -> AcceptingVisitCount play <= cert.rank[start]

checkAbsorption p cert = true
  -> FiniteHorizonIdentitiesAndBounds p cert

checkClosedClass p cert = true
  -> NonterminationProbability p >= PositivePathMass p cert.path

```

The last two require a deliberate staging choice. The matrix checker and finite-horizon bounds can be formalized with rational finite sums first. The path-probability statement requires the probability semantics. Its theorem must not be introduced as an unproved axiom to make the early prototype appear complete. The published state of each theorem should say whether it is specified, proved on paper, implemented in Lean, elaborated, and exercised by an end-to-end source example.

For the game checker, the central reusable lemma is even smaller than the finite arena: if a sequence  $x_n$  has a natural rank strictly decreasing whenever  $B(x_n)$  is false, then  $B(x_n)$  holds infinitely often. A second lemma sums the co-Büchi inequalities. The finite checker need only establish the local hypotheses and closure. This makes the game worker a particularly good first formalization despite

its infinite-play conclusion.

### 6.3 Quoting a certificate without expanding its witness

The generated Lean artifact should define the typed model and certificate as separate constants, prove a concrete Boolean check by kernel reduction or other ordinary proof-producing computation, invoke the general soundness theorem, and then apply the source bridge. Conceptually:

```
private def model : EncodedArena := ...
private def cert : WinCertificate model := ...
private theorem replay : checkWin model cert = true := by
  decide

private theorem model_result := checkWin_sound model cert replay

theorem original_goal := source_bridge model_result
```

This is a deployment template, not a source file reported as compiled. The exact type parameters and declaration syntax should be settled in the pinned Lean project. In particular, a Boolean evaluation outside Lean is not a proof of the displayed replay equation. No new oracle axiom or native-evaluation shortcut is part of the proposed acceptance path.

For pushdown positives, preserve node sharing. A generic soundness proof for a DAG array can induct over its indices and retain semantic summaries, instead of expanding recursive child trees into an explicit run. A final existential reachability theorem does not normally need the concrete transition list to be normalized. If the application explicitly asks to print that list, its exponential output size is unavoidable and must be reported rather than hidden behind a “fast proof” claim.

For game strategies, the source realizer must produce an executable function with the required policy behavior. A finite table can be quoted as a total function on finite indices, with arbitrary legal choices outside the winning region. The source theorem is restricted to starts in the region. For Markov certificates, clear denominator normalizations in a controlled way and prove the exact rational identities; do not make a decimal pretty-printer part of the authoritative path.

### 6.4 Libraries and what they would actually do

| Library or facility                                               | Proposed role and boundary                                                                                                                                                                         |
|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lean core and Mathlib finite data                                 | <b>Nat</b> , <b>Fin</b> , arrays, finite sets, finite sums, and rational arithmetic support the small checkers. Search remains separate; finite data types alone are not checker soundness proofs. |
| Mathlib.<br>Computability.DFA                                     | Reuse DFA evaluation and accepted-language definitions for the regular-target bridge. The inspected definition permits arbitrary state types, so the adapter must also supply finiteness.          |
| Mathlib PMFs and their monad                                      | Give semantics to discrete randomized source programs and compose transition distributions; prove the matrix pushforward equation.                                                                 |
| Mathlib probability kernels and Ionescu–Tulcea trajectory modules | Build or connect to infinite trajectory semantics when the source goal uses path measures or hitting times. These are integration dependencies, not a ready-made absorption checker.               |

| Library or facility               | Proposed role and boundary                                                                                                                                                                                                                 |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FLINT <code>fmpq_mat_solve</code> | Replace the dense Fraction solve with an exact rational search backend. Returned values still face the same residual checker.                                                                                                              |
| Storm                             | Optional search and model-analysis backend for larger probabilistic instances. Use an exact-preserving input/export route and replay against the source-owned rational matrix.                                                             |
| Spot                              | Optional future temporal-monitor producer. Its LTL/ $\omega$ -automata functionality does not imply that an arbitrary exported acceptance condition fits the Büchi checker. Translation and acceptance conversion need their own theorems. |

The DFA module was inspected at Forge’s pinned Mathlib commit `1cf325a0cf67`; its top-first word scan is convenient for the monitor bridge [10]. The other library paths are documented integration points [11, 12, 13, 14, 15, 16], not a claim that a particular collection of imports has been built here. The delivered prototypes require none of those external search libraries.

Using a heavyweight temporal tool should come after the small Büchi lane is working. A deterministic monitor produced from a formula may have a parity or other acceptance condition, not the one accepted by this worker. Silently dropping acceptance colors is not a conversion. The finite round-robin product in Section 3 is a concrete, much smaller next feature.

## 6.5 A router driven by proof obligations

The first router should require explicit registration of a semantic contract. For example, a source lemma can register an empty-stack reachability encoding, a game bridge with its initial state and accepting predicate, or an exact finite probabilistic quotient. Routing then consists of checking the contract’s fragment conditions, quoting finite data, and selecting the corresponding worker. It should not infer a game from a graph while guessing ownership, or a probabilistic chain from numerical weights while guessing normalization.

The returned object can be useful even when it does not finish the entire source theorem. A pushdown nonreachability fact may discharge one branch of an inductive safety proof. A game strategy can supply an existential function and its temporal law. An absorption certificate may prove an exact expected-cost lemma needed by a larger inequality. These are ordinary proved intermediate facts, so the existing controller can use them without a new global proof language.

Relevant resource estimates are available before search:  $n^2a$  potential pushdown summaries,  $n + m$  entries in a game certificate, and the transient matrix dimension and rational bit lengths in a chain. They should control admission and replay budgets separately. The shipped searches are mathematically terminating on finite valid inputs, but do not implement a hardened service scheduler. Resource interruption must return no verdict rather than being interpreted as a negative certificate.

## 6.6 Interface examples, with unsupported features made visible

A future syntax could expose contracts rather than solver switches:

```
forge using pushdown_contract
forge using recurrent_strategy_contract
forge using exact_probability_contract
```

These are proposed front-end forms, not commands installed by this package. Each contract should determine the model, target and bridge. A diagnostic can say “no regular-target bridge was provided,” “the arena has a deadlock,” “an opponent action is missing from the abstraction,” or “the probability rows contain floats.” Such diagnostics identify a missing semantic obligation instead of suggesting that increasing search depth would solve a translation error.

Initial support should be for fixed finite models and fixed rational parameters. A theorem quantified over an arbitrary probability  $p > 0$  may be handled later by a symbolic family theorem, but it is not covered by repeatedly running the finite rational solver at a few sample values. The same restriction applies to arbitrary symbolic stack-frame data and games with an unbounded number of players or states.

## 7 Experiments, replay, and what the evidence does not say

### 7.1 Execution protocol

The recorded run used Python 3.13.5 on Linux, with seed 20260915. All commands ran with `python -S`, which disables automatic site-package loading. The implementation imports only standard-library modules. The source and complete generated certificates are included in the archive, together with the run’s summary and console logs.

The experiments distinguish a *model case*, a *start-state check*, and a *stored certificate record*. A game may have both a nonempty winning region and a nonempty losing region, producing two stored records for one arena. Every vertex is checked as an initial vertex during the experiment, but replay stores one representative start per region. Thus the record count is not a count of distinct theorem problems and certainly not a count of Lean-kernel proofs.

| Family                       | Model cases | Positive records | Negative records |
|------------------------------|-------------|------------------|------------------|
| Pushdown: non-growing        | 500         | 199              | 301              |
| Pushdown: general normalized | 250         | 70               | 180              |
| Pushdown: regular targets    | 100         | 56               | 44               |
| Pushdown: compressed runs    | 31          | 31               | 0                |
| Games: exhaustive sizes 1–3  | 22,100      | 15,388           | 11,260           |
| Games: random sizes 4–7      | 200         | 153              | 114              |
| Chains: acyclic              | 200         | 200              | 0                |
| Chains: cyclic/Cramer oracle | 150         | 150              | 0                |
| Chains: geometric            | 60          | 60               | 0                |
| Chains: reachable traps      | 60          | 0                | 60               |
| Chains: arbitrary four-state | 300         | 250              | 50               |

Table 4: Recorded cases and certificate outcomes. A negative record is an expected mathematical result, not a failed solver invocation. Game records count nonempty regions, not arenas.

Across these datasets the archive contains 28,566 certificate records: 881 pushdown records, 26,915 game-region records, and 770 chain records. All replay successfully. This total describes the archive’s transport workload; it is not a pooled success-rate statistic. No results from the repository’s eighteen historical proposals are included in these numbers.

## 7.2 Pushdown reference computations and their limits

The 500 non-growing systems use rules of replacement length at most one. Their reachable configuration spaces are finite, so concrete breadth-first search from the initial stack is a complete oracle. Its verdict agrees with the summary solver in every case; positive certificates are also expanded into actual runs and checked against the original rules.

The 250 general systems allow pushes of length two. A separate breadth-first search with maximum stack height six finds 70 positive instances and completely exhausts 141 negative instances. In 39 further cases it encounters transitions that exceed its height limit and must report *unknown*. The summary worker returns exclusion certificates for those cases. They pass the independent closure audit, but the bounded concrete oracle does not independently settle them. The run records this limitation rather than counting truncated searches as corroborating negative answers.

The regular-target suite generates non-growing original systems and random three-state target DFAs. It compares a direct predicate-on-stack breadth-first search with the compiled empty-stack query. All 100 verdicts agree. The stored certificate records concern the compiled systems; reproduction of the experiment reconstructs the original target fixtures and performs the source/compiled comparison. Certificate replay alone does not re-prove the compiler bridge.

The compressed family covers depths zero through thirty. All represented lengths equal  $2^{d+1} - 1$  and all positive DAGs have  $d + 1$  nodes. Cases through depth ten are expanded and replayed as concrete traces. Larger cases validate the shared certificate and its length recurrence without materializing the trace. The maximum example’s length is therefore *represented*, not an observed count of executed program steps.

## 7.3 An exhaustive finite-game corpus

For each  $n \in \{1, 2, 3\}$ , the experiment enumerates every nonempty successor subset at each labelled vertex, every binary ownership map, and every accepting subset. The number of arenas is

$$\sum_{n=1}^3 (2^n - 1)^n 2^n 2^n = 4 + 144 + 21,952 = 22,100.$$

Self-loops are allowed. Isomorphic labelled arenas are not deduplicated; the claim is exhaustive coverage of this explicit finite input domain, not a count of nonisomorphic games. An additional 200 arenas have four to seven vertices and at most three distinct outgoing edges per vertex.

The reference oracle does not use attractors or the solver’s rank construction. For each player’s positional policy, it fixes that player’s choices and leaves all opposing choices available. It computes transitive closure by a separate Floyd-style routine. For a protagonist policy it detects reachable cycles in the nonaccepting subgraph. For an antagonist policy it detects reachable cycles containing an accepting vertex. It unions the respective winning sets over all positional policies and checks that they partition the arena. The search’s winning and losing regions agree with this oracle on every arena.

The experiment additionally checks every initial vertex with its region certificate: 67,260 start-state replays, comprising 40,250 winning and 27,010 losing starts. The persistent artifact stores only one representative start per nonempty region. This difference explains why game checks, records, and arenas have different counts.

This is substantially stronger evidence than a handful of successful examples, but it is still finite testing. It shares the standard positional-strategy semantics with the solver and does not replace

the soundness and completeness arguments or their future Lean formalization.

## 7.4 Exact-chain oracles

The 200 acyclic chains are evaluated by backward finite-path dynamic programming. The 150 small cyclic chains have values checked by Cramer’s rule, with determinants computed by Laplace expansion rather than elimination. The 60 geometric chains are checked against the closed forms  $\mathbb{E}\tau = d$ , expected cost  $3d/2$ , and terminal value one for exit probability  $1/d$ . Thus 410 positive cases have numerical values compared against a genuinely different computation or analytic formula.

The 60 explicit trap examples have a known entry probability  $1/d$ , and the returned lower bound agrees exactly. The 300 arbitrary four-state chains use an oracle that enumerates nonempty closed subsets of reachable nonterminal states rather than computing SCCs. It verifies the almost-sure classification, not every numerical field of the positive certificate. All 250 positive and 50 negative classifications agree. Their numerical residuals and path/closure conditions are still checked by the replay module.

This separation is intentional. “Independent oracle” should identify the claim independently checked: exact values in the first three groups, closed-class probability in the fourth, and almost-sure classification in the fifth. It should not be stretched into a statement that every output field was computed twice by unrelated implementations.

## 7.5 Negative controls and transport tests

The 36 named unit tests all pass. They include successful examples as well as invalid-input and certificate mutations, so 36 is not a count of rejected corruptions. Representative controls reject a self-referential pop proof, missing pop bases or binary closure consequences, the wrong pushdown target, an old policy under changed ownership, an omitted opponent self-loop, and a counterrank that cannot pay for an accepting visit.

Probability controls reject nonunit rows, floats, changed transition probabilities, omitted positive-mass successors, a path through a zero-mass edge, a purported trap containing a terminal, and an arbitrary harmonic value with no properness potential. The tests also confirm that an unreachable nonterminal trap does not invalidate absorption from the actual start. Transport controls reject duplicate JSON keys, zero rational denominators, and Python booleans used as integer state identifiers.

These tests exercise real boundaries in the implemented formats. They are not a security audit. The research decoder has an input-size cap, duplicate-key rejection, and rational integer-size checks, but it has not been hardened against every hostile nesting pattern, pathological dimension, or resource exhaustion attack. Unexpected malformed shapes may raise ordinary Python exceptions. A service implementation needs a separately bounded, total decoder and process isolation; none of those are claimed as delivered here.

## 7.6 Search-free replay and performance interpretation

The replay command loads the separately encoded problem, decodes its certificate, calls the appropriate checker, and compares the result with the recorded expected outcome. The checker’s own conclusion is recomputed; the `expected` field is only a regression assertion. Before and after the run, the script checks that the three search modules have not been imported. It prints:

```

Replayed 28566 records; no search modules imported.
reachable: 356                unreachable: 525
winning: 15541               losing: 11374
almost_sure: 660             not_almost_sure: 110

```

Search and replay still share the input model definitions, the codec, and Python’s rational arithmetic implementation. This is more separation than calling the search again during replay, but less than an independently verified checker. Likewise, excluding site packages proves that these runs did not require an installed numerical solver; it does not prove that the code is correct or malicious-input safe.

The summary records internal search, check, and game-oracle timings. The complete experiment loop took approximately 1.62 seconds in the recorded run, excluding JSON serialization, process startup, unit tests, and the later standalone replay. This is a single small-model run, with different numbers of search and replay calls, not a statistical performance study. The timings are retained for reproducibility and troubleshooting, not as a speed claim.

No Lean executable was available in this environment. No Lean source from this package was elaborated, no model theorem was checked by the Lean kernel, and no comparison with `grind`, `Aesop`, `LeanHammer`, or another tactic was run. The historical core Lean elaborations in Forge’s README belong to that repository and are not credited to this work. An end-to-end capability gain in Lean remains a future measurement, although the Python workers already decide their explicitly specified model-level fragments.

## 8 Implementation order and falsifiable acceptance gates

The right next step is not another solver family. It is to turn one of these small replay computations into a checked source theorem. The Büchi rank lane is the first choice: its checker is finite, its core proof uses natural-number descent, and it exposes the important difference between typing a strategy and proving its recurring behavior without initially requiring measure theory.

| Milestone          | Deliverable                                                                                    | Acceptance gate                                                                                                                                                |
|--------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Game re-play    | Lean arena definitions, both rank checkers, and both soundness theorems                        | Compile without placeholders; replay winning and losing fixtures; reject ownership, edge, start and rank mutations; record the actual theorem axiom inventory. |
| 2. Source strategy | One explicit source arena bridge and a generated executable policy                             | Prove the original recurrent specification, not merely the finite table check. A source ownership change must invalidate the bridge or proof.                  |
| 3. Pushdown replay | Productive summary and closed exclusion checkers, suffix theorem, and regular-target reduction | Prove an unbounded-stack safety example and a compressed positive reachability example without expanding their configuration spaces or traces.                 |

| Milestone             | Deliverable                                                                    | Acceptance gate                                                                                                                             |
|-----------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 4. Rational residuals | Stochastic-matrix validator and finite-horizon identities and bounds           | Check the retry example and reject the harmonic-only self-loop. Verify costs and fractional terminal payoffs under their distinct meanings. |
| 5. Probability bridge | A discrete source semantics linked to the matrix by exact pushforward          | Prove almost-sure termination and the original expected-cost statement; changing an original probability must invalidate the proof.         |
| 6. Tactic integration | Contract recognition, search invocation, cached replay and source-goal closure | Evaluate against pinned baseline tactics and manually supplied model proofs, with model construction and kernel replay costs included.      |

The comparison at milestone 6 should separate two sources of value. One track supplies the same source bridge and local lemmas to all tactics, testing how well the new worker finds the model certificate. A second track measures the whole user experience, including contract recognition and bridge construction. The first must not be described as an end-to-end automation result; the second must not hide manually authored bridges as if the tactic synthesized them. The existing repository’s general benchmark discipline can be reused; only these worker-specific distinctions need to be added.

## 8.1 What should remain outside the first release

General parity games, games with imperfect information, arbitrary temporal formulas, stochastic games, unbounded-state Markov chains, and arbitrary recursive Lean definitions should remain explicitly unsupported. The small round-robin monitor, queue-based attractors, exact finite quotients, and fraction-free linear solving are concrete extensions with clear proof obligations. They are better next targets than a claim to solve all of the larger subjects at once.

The three workers also cannot be combined merely by connecting their data formats. An existential pop summary does not summarize an adversarial stack game: its successful internal choices might belong to the opponent. A probabilistic pushdown process can produce nonlinear probability equations and is not automatically a finite rational Markov chain. Combining the present workers requires a proved intermediate semantics, not a shared use of words such as “state,” “closure,” or “transition.”

## 8.2 Assessment

These extensions expand the kinds of statements Forge could automate rather than only increasing a saturation budget. They give finite witnesses for unbounded stack behavior, persistent strategies for infinite adversarial execution, and properness-backed exact values for stochastic execution. Their negative results have equally concrete meanings: a closed exclusion relation, a counterstrategy bounding accepting visits, or a positive-probability path to a nonterminal closed class.

The strongest completed deliverable is the combination of executable searches, separate replay code, explicit source-bridge contracts, and exhaustive or orthogonal tests on carefully delimited fragments. The principal unfinished work is the Lean port and its end-to-end source examples. That work is now a set of named checker theorems and translation obligations, rather than a request to design a more powerful prover in the abstract.

## A Wire format and executable entry points

The transport uses JSON with explicit rational objects:

```
{"rational": [numerator, positive_denominator]}
```

Integer identifiers, rational values and absent entries are distinct. A vector entry outside a certified region is `null`, not an arbitrary numerical zero. A record contains **problem**, **certificate**, and **expected**; a game record also supplies its **start**. Only the first two and the game start determine the check. The expected result is a test assertion, not a premise of the mathematical certificate.

| Certificate kind         | Principal fields                                                                              | Meaning of acceptance                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>pds_reach</code>   | <b>nodes</b> : triple, input rule index, earlier children; <b>chain</b> : initial-stack roots | An actual finite run empties the stack at an input final control.           |
| <code>pds_exclude</code> | <b>relation</b> : distinct summary triples                                                    | A rule-closed relation excludes every root chain to an input final control. |
| <code>buchi_win</code>   | <b>region</b> , <b>rank</b> , <b>policy</b>                                                   | One protagonist policy forces recurrent acceptance from the supplied start. |
| <code>buchi_lose</code>  | <b>region</b> , <b>rank</b> , <b>policy</b>                                                   | One antagonist policy bounds all accepting visits from the supplied start.  |
| <code>mc_absorb</code>   | <b>support</b> , <b>time</b> , <b>cost</b> , <b>value</b>                                     | Properness and exact time, cost, and terminal-payoff equations hold.        |
| <code>mc_trap</code>     | <b>closed</b> , <b>path</b>                                                                   | A positive-probability source path enters a closed nonterminal set.         |

The Python APIs are `pushdown.solve(p)`, `games.solve(g)`, and `markov.solve(c)`. The game solver returns both regions; the caller chooses the certificate whose region contains the desired start. The replay APIs are `check_pds(p, cert)`, `check_game(g, cert, start)`, and `check_chain(c, cert)`. They validate the input again. Their checked models are immutable dataclasses, but the research certificate dictionaries are not unforgeable authority objects; only the eventual Lean proof could supply that authority.

## B Reproduction and artifact map

From the extracted archive's top directory:

```
python -S -m unittest discover -s tests -v
python -S prototype/demo.py
python -S prototype/verify.py
python -S prototype/run_experiments.py
python -S prototype/verify.py reproduced-results/certificates.json
```

The default experimental output is `reproduced-results/`, so a new run does not overwrite the recorded evidence. The `-quick` option restricts exhaustive games to sizes one and two; it is a

different corpus and will not reproduce the full game counts. Do not invoke the tests with Python's optimized `-O` flag, since reference-oracle assertions are part of the experiment.

Build the article with stock pdfLaTeX:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge-infinite-horizon.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-infinite-horizon.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-infinite-horizon.tex
```

No bibliography program, shell escape, downloaded font, or numerical package is required. The archive includes the master source, section sources, references, and the rendered PDF.

`results/certificates.json` stores all certificate records; `results/summary.json` records the machine-readable counts and timings; `results/unit-tests.txt`, `results/replay.txt`, and `results/demo.txt` retain console evidence. `tests/oracles.py` contains the alternative computations. `docs/STATUS.md` distinguishes executed code from proposed Lean work, and `docs/SOURCES.md` records the repository and library provenance.

## C A compact theorem-obligation ledger

| Object                  | Mathematical obligation                                                    | Delivered evidence                                                                          |
|-------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Pushdown summaries      | Productive closure is exact; suffix lifting preserves untouched stack data | Paper proofs; agenda search; local replay; finite and bounded concrete oracles              |
| Regular target compiler | Target language iff compiled empty-stack target                            | Paper proof; 100 complete source/compiled comparisons; orientation and empty-stack controls |
| Positive game rank      | Every policy-consistent play visits acceptance infinitely often            | Paper proof; search; rank replay; exhaustive small arenas and policy-enumeration oracle     |
| Negative game rank      | Every counterpolicy-consistent play has bounded accepting visits           | Paper proof; dual search; rank replay; same exhaustive corpus                               |
| Absorption values       | Properness plus residuals imply exact values and finite-horizon bounds     | Paper proof; exact search and replay; numerical and graph reference computations            |
| Closed-class path       | Positive path mass lower-bounds non-termination probability                | Paper proof; SCC search; local replay; analytic and closed-subset controls                  |
| Lean and source bridges | Accepted object theorem implies the original typed source proposition      | Detailed contracts and implementation plan only; no compiled Lean artifacts                 |

## References

- [1] Vladimir Reshetnikov, *Forge*, README and published project status, commit `c98e47c5f804e92880fc1d0e378c1b95832b685c`. Inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/tree/c98e47c5f804e92880fc1d0e378c1b95832b685c>.
- [2] Forge contributors, *Provenance: what came from where*, Appendix B of the merged article, at the same commit. The ledger maps all eighteen contributing proposals to their retained capabilities. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/B-provenance.tex>.
- [3] Forge contributors, *Assessment and recommended starting point*, merged article conclusion, at the same commit. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/12-conclusion.tex>.
- [4] Forge contributors, *Ideas from Leant and Djex*, integration review, at the same commit. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] Vladimir Reshetnikov and contributors, *Leant: a Djex-based synthesis REPL for Lean 4*. README, inspected 15 September 2026. A live interface description, not a reproduction of the project’s acceptance runs. <https://github.com/VladimirReshetnikov/Leant/blob/main/README.md>.
- [6] Vladimir Reshetnikov and contributors, *Djex*, README, first 140 lines inspected 15 September 2026; inspected README blob `12fec0edf075937f2eb42125e800845a480d4b76`. <https://github.com/VladimirReshetnikov/Djex/blob/main/README.md>.
- [7] Arnaud Carayol and Matthew Hague, “Saturation algorithms for model-checking pushdown systems,” *Electronic Proceedings in Theoretical Computer Science* 151 (2014), 1–24. DOI: 10.4204/EPTCS.151.1. <https://arxiv.org/abs/1405.5593>. This is the reference for the established saturation framework, not a claim that the present certificate packaging is a new reachability algorithm.
- [8] Krishnendu Chatterjee and Monika Henzinger, *An  $O(n^2)$  Time Algorithm for Alternating Büchi Games*, Technical Report IST-2011-0009, 11 July 2011. The preliminaries describe positional determinacy and the classical repeated-attractor algorithm. The faster algorithm named in the title is not implemented in this package. [https://research-explorer.ista.ac.at/download/5379/5504/IST-2011-0009\\_IST-2011-0009.pdf](https://research-explorer.ista.ac.at/download/5379/5504/IST-2011-0009_IST-2011-0009.pdf).
- [9] Lean project, *The grind tactic*, Lean language reference, inspected 15 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [10] Mathlib contributors, `Mathlib.Computability.DFA`, source inspected at commit `1cf325a0cf67aca2b04d76b5380ff6a9e410aefa`, the Mathlib baseline named by Forge. Includes `DFA`, `DFA.evalFrom`, and word evaluation. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/Mathlib/Computability/DFA.lean>.
- [11] Mathlib contributors, *Probability mass functions*, documentation for `Mathlib.Probability.ProbabilityMassFunction.Basic` and

- `Mathlib.Probability.ProbabilityMassFunction.Monad`, inspected 15 September 2026.  
[https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Probability/ProbabilityMassFunction/Basic.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Probability/ProbabilityMassFunction/Basic.html). [https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Probability/ProbabilityMassFunction/Monad.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Probability/ProbabilityMassFunction/Monad.html).
- [12] Mathlib contributors, *Markov kernels*, `Mathlib.Probability.Kernel.Defs`, documentation inspected 15 September 2026. [https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Probability/Kernel/Defs.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Probability/Kernel/Defs.html).
- [13] Mathlib contributors, documentation index and Ionescu–Tulcea trajectory modules, including `Mathlib.Probability.Kernel.IonescuTulcea.Traj`. Module availability inspected 15 September 2026; no import-level build is claimed here.  
[https://leanprover-community.github.io/mathlib4\\_docs/Mathlib](https://leanprover-community.github.io/mathlib4_docs/Mathlib).
- [14] FLINT contributors, *Matrices over the rational numbers*, `fmpq_mat.h` documentation, especially `fmpq_mat_solve`. Inspected 15 September 2026.  
[https://flintlib.org/doc/fmpq\\_mat.html](https://flintlib.org/doc/fmpq_mat.html).
- [15] Storm developers, *Running Storm*, official usage documentation, sections on exact computation and explicit models, inspected 15 September 2026.  
<https://www.stormchecker.org/documentation/usage/running-storm.html>.
- [16] Spot developers, *Spot: a platform for LTL and  $\omega$ -automata manipulation*, official project description, inspected 15 September 2026. <https://spot.lre.epita.fr/>.